

# tf.OptionalSpec Stay organized with collections Save and categorize content based on your preferences.

[https://www.tensorflow.org/api\\_docs/python/tf/OptionalSpec](https://www.tensorflow.org/api_docs/python/tf/OptionalSpec)

---

*Type specification for tf.experimental.Optional.*

## Code Examples

---

```
tf.OptionalSpec(  
    element_spec  
)
```

```
tf.OptionalSpec(  
    element_spec  
)
```

```
@tf.function(input_signature=[tf.OptionalSpec(  
    tf.TensorSpec(shape=(), dtype=tf.int32, name=None))])  
def maybe_square(optional):  
    if optional.has_value():  
        x = optional.get_value()  
        return x * x  
    return -1  
optional = tf.experimental.Optional.from_value(5)  
print(maybe_square(optional))  
tf.Tensor(25, shape=(), dtype=int32)
```

```
@tf.function(input_signature=[tf.OptionalSpec(
```

```
tf.TensorSpec(shape=(), dtype=tf.int32, name=None))])
```

```
def maybe_square(optional):
```

```
    if optional.has_value():
```

```
        x = optional.get_value()
```

## Documentation

---

Type specification for `tf.experimental.Optional`.

Inherits From: `TypeSpec`, `TraceType`

### View aliases

Compat aliases for migration

See

Migration guide for  
more details.

`tf.compat.v1.OptionalSpec`, `tf.compat.v1.data.experimental.OptionalStructure`

For instance, `tf.OptionalSpec` can be used to define a `tf.function` that takes

tf.experimental.Optional as an input argument:

## Attributes

In particular, all values that are compatible with this TypeSpec must be an instance of this type.

## Methods

`## experimental_as_proto`

[View source](#)

Returns a proto representation of the TypeSpec instance.

Do NOT override for custom non-TF types.

### **experimental\_from\_proto**

[View source](#)

Returns a TypeSpec instance based on the serialized proto.

Do NOT override for custom non-TF types.

### **experimental\_type\_proto**

[View source](#)

Returns the type of proto associated with TypeSpec serialization.

Do NOT override for custom non-TF types.

### **from\_value**

[View source](#)

## **is\_compatible\_with**

[View source](#)

Returns true if spec\_or\_value is compatible with this TypeSpec.

Prefer using "is\_subtype\_of" and "most\_specific\_common\_supertype" wherever possible.

## **is\_subtype\_of**

[View source](#)

Returns True if self is a subtype of other.

Implements the tf.types.experimental.func.TraceType interface.

If not overridden by a subclass, the default behavior is to assume the TypeSpec is covariant upon attributes that implement TraceType and invariant upon rest of the attributes as well as the structure and type of the TypeSpec.

## **most\_specific\_common\_supertype**

[View source](#)

Returns the most specific supertype TypeSpec of self and others.

Implements the tf.types.experimental.func.TraceType interface.

If not overridden by a subclass, the default behavior is to assume the `TypeSpec` is covariant upon attributes that implement `TraceType` and invariant upon rest of the attributes as well as the structure and type of the `TypeSpec`.

## **`most_specific_compatible_type`**

[View source](#)

Returns the most specific `TypeSpec` compatible with self and other. (deprecated)

Deprecated. Please use `most_specific_common_supertype` instead.

Do not override this function.

## **`__eq__`**

[View source](#)

Return `self==value`.

## **`__ne__`**

[View source](#)

Return `self!=value`.